

AWS EKS Cluster Automation via eksctl

Comprehensive Infrastructure-as-Code Deployment Log & Engineering Manual

Date:	June 4, 2026	Primary Engine:	eksctl CLI (v0.227.0)
Target Region:	Mumbai, India (ap-south-1)	Orchestrator Path:	MobaXterm POSIX Subsystem
Engineer Profile:	Mahfuz (Dhaka Client)	Deployment State:	Active / Verified

1. Prerequisites & Host Configuration

Before initializing cluster provisioning, ensure that your local terminal environment is completely authenticated with administrative cloud access credentials. The deployment steps assume that your AWS profile variables are already bound and configured.

```
# Verify local AWS CLI integration and version profiles
aws --version

# Bind regional access parameters and active credential keys
aws configure
```

2. Environment Setup & Tool Installation

Initially, native operations to set up package runtimes using macOS/Linux standards (brew) or native Windows orchestration variants (winget, choco) triggered continuous system shell faults (command not found). This block occurred because the workspace context was executing within an isolated local POSIX subsystem wrapper emulation layer inside MobaXterm.

To eliminate environment routing mismatches, the target standalone compilation executable binary was securely extracted directly from the official system package release indexes, mapped, and linked manually into the runtime profile PATH variable:

```
# 1. Pull the target compiled Windows AMD64 release zip package archive
curl --silent --location "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Windows_amd64.zip" -o "eksctl.zip"

# 2. Unzip and extract the core executable binary dependencies
unzip eksctl.zip

# 3. Clean up the leftover archive package to maintain a clean workspace footprint
rm eksctl.zip

# 4. Move the compiled executable to your localized system bin for global access
mv eksctl.exe /bin/

# 5. Confirm environment path visibility and executable version status
eksctl version
```

3. Infrastructure Deployment (EKS Provisioning)

Utilizing the configured eksctl engine, a deployment configuration call was issued to spin up an enterprise-grade, managed Kubernetes environment mapped to the Mumbai (ap-south-1) geographic deployment target—providing the minimum network latency lines for clients routing from Dhaka, Bangladesh.

```
# Automated declarative cluster creation sequence
eksctl create cluster \
  --name mumbai-eks-cluster \
  --region ap-south-1 \
  --nodegroup-name standard-workers \
  --node-type t3.medium \
  --nodes 2 \
  --managed
```

Behind the scenes, the wrapper maps this command into nested AWS CloudFormation configuration templates, handling the lifecycle automation of several dependencies: a dedicated Virtual Private Cloud (VPC) topology with public/private isolated subnet pairs across multiple Availability Zones, the AWS-managed Kubernetes API Control Plane, and a Managed Node Group of 2 EC2 worker nodes shielded inside private subnets (which is why direct host network pings fail by architectural security design).

4. Post-Deployment Cluster Verification

Once the engine returns an infrastructure status of ready, the cluster context mapping file (~/.kube/config) updates automatically. Run the validation checks via the Kubernetes command-line tool (kubectl) to analyze node status profiles:

```
kubectl get nodes
```

Verified computing layer allocation table returned directly from the API control plane:

NODE INTERNAL DNS NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-20-148.ap-south-1.compute.internal	Ready	<none>	11m	v1.34.8-eks-3385e9b
ip-192-168-64-117.ap-south-1.compute.internal	Ready	<none>	11m	v1.34.8-eks-3385e9b

5. Next Steps: Workload Deployment

To confirm complete operational validation of the container scheduler engine, map an application benchmark service onto your compute node structures:

```
# 1. Initialize an Nginx web application micro-container deployment
kubectl create deployment nginx-test --image=nginx

# 2. Check the scheduling loop state until the container reads 'Running'
kubectl get pods

# 3. Audit cluster-wide baseline addon microservices
kubectl get pods -n kube-system
```

6. Teardown & Lifecycle Management (Cost Prevention)

To eliminate cloud infrastructure resource drift and optimize your AWS spending metrics when testing tasks conclude, always execute the programmatic decommissioning pipeline. This will systematically terminate all nested node groups, network stacks, and active master planes safely:

```
eksctl delete cluster --name mumbai-eks-cluster --region ap-south-1
```